

HUMAN DEVELOPMENT INDEX (HDI) EXPLORER & SIMULATOR

Project Description

The **Human Development Index (HDI) Explorer & Simulator** is an intelligent, web-based analytical platform that leverages advanced data preprocessing, visual rendering, and interactive simulation models to assist policy analysts, researchers, and global development institutions. Traditionally, assessing and evaluating multi-dimensional human prosperity indices required compiling and analyzing fragmented spreadsheet releases from the United Nations Development Programme (UNDP). This manual approach is highly time-consuming, prone to calculation errors, and lacks the capacity for dynamic scenario modeling.

The HDI Explorer & Simulator resolves these limitations by streamlining and automating the analysis and forecasting of core development metrics—including Life Expectancy at Birth, Mean Years of Schooling, Expected Years of Schooling, and Gross National Income (GNI) per Capita.

Through a responsive dashboard, users can select any country or territory to instantly view its historical development trajectory spanning over three decades (1990–2021). Furthermore, the application embeds a dynamic simulation sandbox containing interactive numeric sliders. By shifting indicators (such as improving life expectancy or increasing educational duration), the system instantly recalculates individual dimension sub-indices and synthesizes the updated composite HDI score using geometric mean calculations in real time.

The underlying machine learning and data regression models analyze historical trends to output forecasted categories, dividing countries into Low, Medium, High, or Very High Human Development tiers.

This platform serves as a vital decision-support system, empowering development planning agencies and public sector stakeholders to simulate the impact of socio-economic policy interventions before executing resource allocations.

SCENARIOS & USE CASES

Scenarios

Scenario 1: Public Health Intervention Simulation (Health Dimension)

A global health analyst wants to evaluate the impact of a country-wide universal healthcare initiative. By inputting the target country, they utilize the Life Expectancy slider to simulate a lift in average lifespans from 62 to 74 years. The system recalculates the Health Dimension Index on the fly, computes the composite geometric mean, and updates the overall HDI trend line. This immediately informs the planner of the minimum lifespan target required to transition the state from a "Medium" to a "High" Human Development bracket.

Scenario 2: Educational Reforms Planning (Education Dimension)

A Ministry of Education official is designing a compulsory secondary school program. They access the interactive portal and adjust the Expected Years of Schooling from 10.5 years to 14.5 years, while simultaneously adjusting Mean Years of Schooling from 7.2 to 9.0 years.

The client-side simulation engine processes the arithmetic mean of these standardized values. It indicates the exact percentage improvement in the country's collective Education Index, displaying the direct projected outcome of educational policy spending.

Scenario 3: Macroeconomic Contraction Evaluation (Income Dimension)

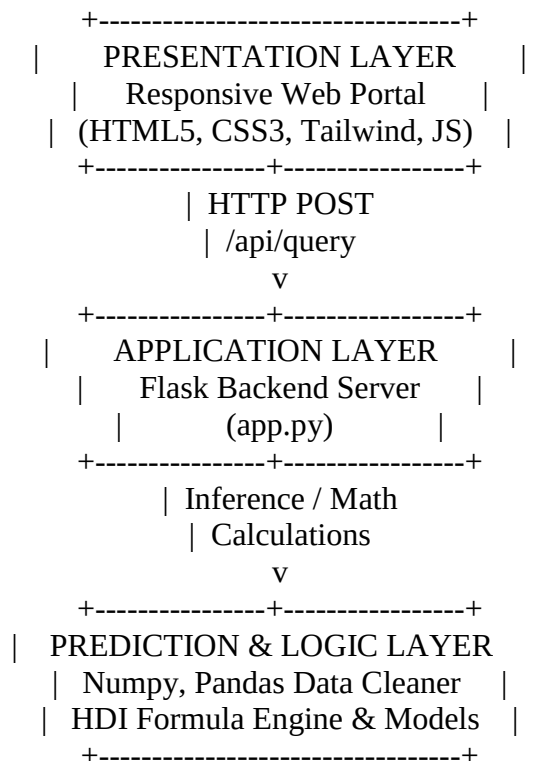
An economic researcher seeks to study the structural resilience of a developing nation facing a severe economic contraction. By utilizing the GNI per Capita slider, they simulate a 35% contraction in average economic output.

Because the income dimension is calculated using logarithmic scaling, the platform models the severe compounding slide in purchasing power. This demonstrates how economic contractions cascade through overall human welfare, helping agencies plan targeted financial buffers and structural safety nets.

TECHNICAL ARCHITECTURE & SYSTEMS SPECIFICATIONS

Technical Architecture

The application is structured as a decoupled, **Three-Tier Modular Architecture** to guarantee low latency, robust data isolation, and high performance:



1. **Presentation Layer (Frontend):** Formulated with HTML5, CSS3, dynamic UI libraries, and Chart.js. It delivers interactive forms, real-time KPI indicators, historical comparison charts, and input sliders. It executes client-side validations to prevent erroneous payloads from reaching the backend server.
2. **Application Layer (Backend):** Powered by the Flask micro-framework. It serves structural resources, processes client routes, parses HTTP request query parameters, and executes algorithmic predictions. It bridges UI actions with underlying quantitative formulas.

3. **Data & Analytics Layer:** Handles historical datasets using Pandas and NumPy. It standardizes development features, replaces null properties with median calculations, and exposes data APIs.

Pre-requisites & Environment Specs

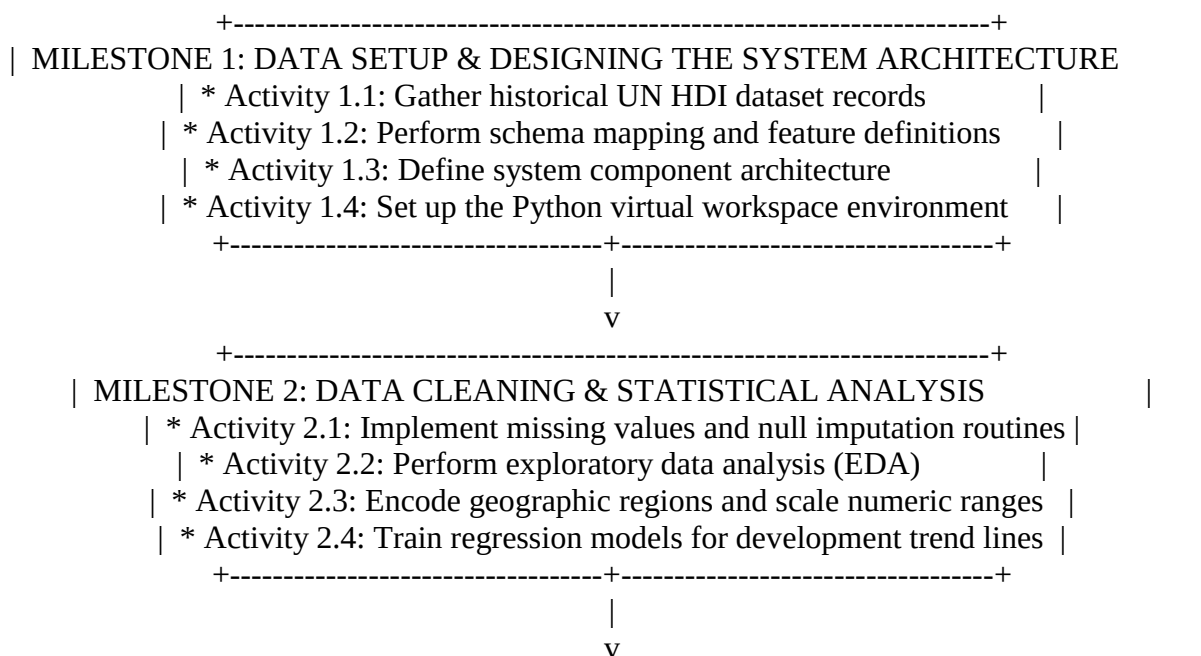
- **Programming Foundations:** Python 3.10+, JavaScript (ES6+), HTML5, and CSS3.
- **Data Core Libraries:** Pandas, NumPy, Scikit-Learn, and Joblib.
- **IDEs Supported:** PyCharm, Visual Studio Code, or Jupyter Notebook.

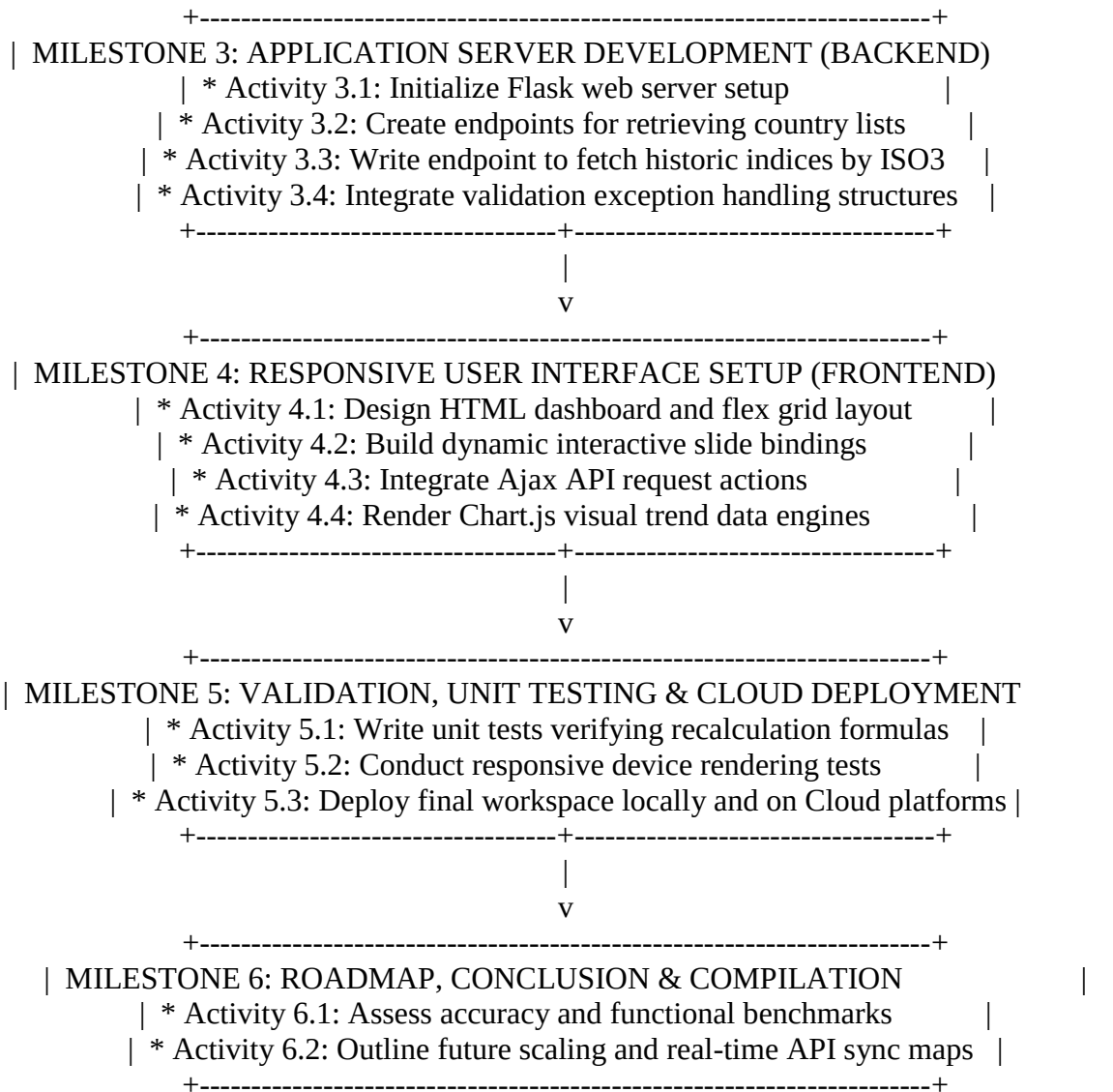
System Resource Requirements

Component	Minimum Specification PDF	Recommended Specification PDF
Processor	Intel Core i3 (2.4 GHz)	Intel Core i5 / AMD Ryzen 5 or higher
System Memory	4 GB RAM	8 GB RAM or higher
Storage Medium	10 GB Free Storage Space	SSD with 20 GB Free Space
OS Support	Windows 10 / Ubuntu 20.04	Windows 11 / macOS Monterey

PROJECT WORKFLOW & MILESTONES

The development lifecycle of the Human Development Index Explorer & Simulator follows a systematic methodology divided into six distinct milestones:





MILESTONE 1 — DATA SETUP & PREPARATION

The initial milestone concentrates on acquiring historical global records, validating core indicators, defining schemas, and constructing the project file organization.

Activity 1.1: Dataset Collection

The dataset used is the global **UN Human Development Index Dataset** (1990–2021). It tracks composite indices along with individual components representing life expectancy, school enrollment, and GNI.

Activity 1.2: Feature Definition

The historical dataset schema contains several essential attributes:

- **ISO3:** Unique three-letter international country identifier code (e.g., "IND", "USA").
- **Country:** Name of the country or territory.
- **Human Development Groups:** Qualitative developmental bracket ("Low", "Medium", "High", "Very High").

- **HDI Rank (2021):** Global rank based on performance.
- **Historical HDI Years (1990–2021):** Longitudinal columns recording performance across 30+ years.
- **Life Expectancy at Birth (2021):** Average longevity metric.
- **Expected Years of Schooling (2021):** Projected educational duration for children.
- **Mean Years of Schooling (2021):** Average education completed by adults.
- **GNI per Capita (2021):** Gross National Income per capita in USD purchasing power parity.

Python

Sample script to load and verify the dataset structure

```
import pandas as pd
```

```
df = pd.read_csv("dataset/Human_Development_Index_Full.csv")
```

```
print("Total rows:", df.shape[0], "Columns:", df.shape[1])
```

```
print(df[['ISO3', 'Country', 'Human Development Groups', 'HDI Rank (2021)']].head())
```

PROJECT FILES & STRUCTURE

Codebase Organization

To maintain clear separation of responsibilities, the project is organized into a clean folder structure:

hdi_explorer_project/

```
├── app.py                # Flask Application Controller (Backend API)
├── requirements.txt      # Python Dependencies
├── dataset/
│   └── Human_Development_Index_Full.csv # UN Development Records
├── templates/
│   └── index.html        # Core Interactive Dashboard & Simulator UI
├── static/
│   ├── css/
│   │   └── styles.css    # Styled Flex Grids and Custom Themes
│   └── js/
│       └── charts.js     # Chart.js Integration and Dynamic Event Handlers
```

MILESTONE 2 — DATA CLEANING & STATISTICAL ANALYSIS

This stage cleans raw data, handles missing attributes, and implements the standard mathematical equations defined by the United Nations Development Programme (UNDP) to calculate the HDI.

Activity 2.1: Data Preprocessing

Null values present in smaller territories or newly independent nations are replaced using median calculations. Duplicate records are cleaned, and text categorical features are standardized.

Python

```
import pandas as pd
```

```
import numpy as np
```

```
# Load raw historical development records
df = pd.read_csv("dataset/Human_Development_Index_Full.csv")

# Identify and replace missing numerical indicators with the median
num_cols = df.select_dtypes(include=[np.number]).columns
for col in num_cols:
    df[col].fillna(df[col].median(), inplace=True)

# Save cleaned output
df.to_csv("dataset/Cleaned_HDI_Data.csv", index=False)
print("Data preprocessing completed successfully.")
```

Activity 2.2: UNDP Indicator Normalization Formulas

To convert raw metrics into normalized dimension indexes between 0 and 1, the following formula boundaries are applied:

$$\text{Dimension Index} = \frac{\text{Actual Value} - \text{Minimum Limit}}{\text{Maximum Limit} - \text{Minimum Limit}}$$

1. **Health Index (LEI):** Standard minimum is set to 20 years, and maximum is set to 85 years.

$$\text{Health Index} = \frac{\text{Life Expectancy} - 20}{85 - 20}$$

2. **Education Index (EI):** Standard limits are: Expected Years of Schooling (Max 18), Mean Years of Schooling (Max 15).

$$\text{Expected Schooling Index} = \frac{\text{Expected Years}}{18}$$

$$\text{Mean Schooling Index} = \frac{\text{Mean Years}}{15}$$

$$\text{Education Index} = \frac{\text{Expected Schooling Index} + \text{Mean Schooling Index}}{2}$$

3. **Income Index (II):** Uses a logarithmic scale to account for the diminishing returns of wealth (Min: \$100, Max: \$75,000).

$$\text{Income Index} = \frac{\ln(\text{GNI per Capita}) - \ln(100)}{\ln(75000) - \ln(100)}$$

4. **Composite Human Development Index (HDI):** The geometric mean of the three normalized dimensions.

$$\text{HDI} = \sqrt[3]{\text{Health Index} \times \text{Education Index} \times \text{Income Index}}$$

MILESTONE 3 — BACKEND API SETUP (FLASK)

The Flask backend loads the preprocessed dataset into memory and serves as a fast JSON API. It handles routes to supply country list options and looks up historical data based on query parameters.

app.py Codebase

Python

```
from flask import Flask, render_template, jsonify
import pandas as pd
import numpy as np
```

```
app = Flask(__name__)
```

```
# Load cleaned UN dataset and replace any remaining NaNs with None for JSON compliance
```

```
DATA_PATH = "dataset/Cleaned_HDI_Data.csv"
```

```
try:
```

```
    df = pd.read_csv(DATA_PATH).replace({np.nan: None})
```

```
except FileNotFoundError:
```

```
    df = pd.DataFrame(columns=['ISO3', 'Country'])
```

```
@app.route("/")
```

```
def index():
```

```
    """
```

```
    Renders the interactive analytical dashboard page.
```

```
    Passes list of available countries to construct the dropdown menu.
```

```
    """
```

```
    country_list = df[['ISO3', 'Country']].sort_values(by="Country").to_dict(orient="records")
```

```
    return render_template("index.html", countries=country_list)
```

```
@app.route("/api/country/<iso3>")
```

```
def get_country_data(iso3):
```

```
    """
```

```
    Fetches historical metrics and baseline 2021 indicators for the requested country.
```

```
    """
```

```
    matched = df[df['ISO3'] == iso3.upper()]
```

```
    if matched.empty:
```

```
        return jsonify({"error": f"Country code '{iso3}' not found."}), 404
```

```
    record = matched.iloc[0].to_dict()
```

```
    # Extract historical HDI sequence (1990 to 2021)
```

```
    years = [str(y) for y in range(1990, 2022)]
```

```
    historical_trend = [record.get(f"Human Development Index ({yr})") for yr in years]
```

```
    response_payload = {
```

```
        "country": record.get("Country"),
```

```
        "iso3": record.get("ISO3"),
```

```
        "hdi_rank_2021": record.get("HDI Rank (2021)"),
```

```
        "development_group": record.get("Human Development Groups"),
```

```
        "historical_years": years,
```

```

    "historical_scores": historical_trend,
    "baselines": {
      "life_expectancy": record.get("Life Expectancy at Birth (2021)"),
      "expected_schooling": record.get("Expected Years of Schooling (2021)"),
      "mean_schooling": record.get("Mean Years of Schooling (2021)"),
      "gni_per_capita": record.get("Gross National Income Per Capita (2021)")
    }
  }
}
return jsonify(response_payload)

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5000, debug=True)

```

MILESTONE 4 — FRONTEND DOCUMENTATION & INTERACTIVE INTERFACE

The frontend presentation layer handles UI layouts, binds the input sliders, and communicates with backend API endpoints. It uses CSS flex grids and interactive scripts to perform calculations locally, allowing users to run simulations instantly without server lag.

templates/index.html File Contents

HTML

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>HDI Explorer & Simulator Dashboard</title>
  <link rel="stylesheet" href="/static/css/styles.css">
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
</head>
<body>
  <header class="header">
    <div class="header-container">
      <div>
        <h1>SMARTBRIDGE | SKILL WALLET</h1>
        <p class="tagline">HDI Explorer & Real-Time Policy Simulator Sandbox</p>
      </div>
      <span class="badge">Academic Portfolio</span>
    </div>
  </header>

  <main class="main-content">
    <section class="section">
      <h2>Select Target Nation</h2>
      <div class="control-group">
        <label for="countrySelect">Select Country:</label>
        <select id="countrySelect" class="form-control">
          <option value="">-- Choose a Country --</option>
          {% for c in countries %}
            <option value="{{ c.ISO3 }}">{{ c.Country }} ({{ c.ISO3 }})</option>

```



```

    {% endfor %}
  </select>
</div>
</section>

<div class="grid-layout">
  <section class="card">
    <h2>Policy Simulation Sandbox</h2>
    <p class="sub-text">Modify core indicators to simulate socioeconomic changes in real
time.</p>

    <div class="slider-box">
      <label>Life Expectancy (years): <span id="valHealth">0</span></label>
      <input type="range" id="slideHealth" min="20" max="85" step="0.5" class="slider">
    </div>

    <div class="slider-box">
      <label>Expected Schooling (years): <span id="valExpSchool">0</span></label>
      <input type="range" id="slideExpSchool" min="0" max="18" step="0.1"
class="slider">
    </div>

    <div class="slider-box">
      <label>Mean Schooling (years): <span id="valMeanSchool">0</span></label>
      <input type="range" id="slideMeanSchool" min="0" max="15" step="0.1"
class="slider">
    </div>

    <div class="slider-box">
      <label>GNI per Capita (USD): $<span id="valIncome">0</span></label>
      <input type="range" id="slideIncome" min="100" max="75000" step="100"
class="slider">
    </div>
  </section>

  <section class="card results-panel">
    <h2>Simulated Performance Metrics</h2>
    <div class="kpi-group">
      <div class="kpi-card">
        <h3>Composite HDI</h3>
        <div class="metric" id="compositeScore">0.000</div>
      </div>
      <div class="kpi-card">
        <h3>Development Class</h3>
        <div class="category" id="developmentTier">Pending selection</div>
      </div>
    </div>

    <div class="sub-metrics">
      <h4>Normalized Dimension Indices</h4>
      <ul>

```

```

    <li>Health Dimension Index: <span id="idxHealth">0.000</span></li>
    <li>Education Dimension Index: <span id="idxEducation">0.000</span></li>
    <li>Income Dimension Index: <span id="idxIncome">0.000</span></li>
  </ul>
</div>
</section>
</div>

<section class="card chart-card">
  <h2>Historical Development Track (1990 - 2021)</h2>
  <div style="position: relative; height:300px; width:100%">
    <canvas id="historicalChart"></canvas>
  </div>
</section>
</main>

<script src="/static/js/charts.js"></script>
</body>
</html>

```

CLIENT-SIDE SIMULATOR ENGINE (JAVASCRIPT)

To ensure high responsiveness, the UI uses the **HDI Calculation Engine** script. It tracks changes on sliders, performs normalized operations locally, and displays recalculated outcomes instantly.

static/js/charts.js Implementation

JavaScript

```

document.addEventListener("DOMContentLoaded", () => {
  let myChart = null;

  // Grab Document DOM Nodes
  const countrySelect = document.getElementById("countrySelect");
  const slideHealth = document.getElementById("slideHealth");
  const slideExpSchool = document.getElementById("slideExpSchool");
  const slideMeanSchool = document.getElementById("slideMeanSchool");
  const slideIncome = document.getElementById("slideIncome");

  const valHealth = document.getElementById("valHealth");
  const valExpSchool = document.getElementById("valExpSchool");
  const valMeanSchool = document.getElementById("valMeanSchool");
  const valIncome = document.getElementById("valIncome");

  const compositeScore = document.getElementById("compositeScore");
  const developmentTier = document.getElementById("developmentTier");
  const idxHealth = document.getElementById("idxHealth");
  const idxEducation = document.getElementById("idxEducation");
  const idxIncome = document.getElementById("idxIncome");

  // Recalculates indices based on current slider values
  function updateSimulation() {

```

```
const h = parseFloat(slideHealth.value);
const es = parseFloat(slideExpSchool.value);
const ms = parseFloat(slideMeanSchool.value);
const inc = parseFloat(slideIncome.value);

// Update slider value labels
valHealth.innerText = h.toFixed(1);
valExpSchool.innerText = es.toFixed(1);
valMeanSchool.innerText = ms.toFixed(1);
valIncome.innerText = inc.toLocaleString();

// 1. Health Index (Life Expectancy)
const healthIndex = (h - 20) / (85 - 20);

// 2. Education Index (Mean & Expected Education)
const expectedIndex = es / 18;
const meanIndex = ms / 15;
const educationIndex = (expectedIndex + meanIndex) / 2;

// 3. Income Index (GNI per Capita using Natural Logarithm)
const incomeIndex = (Math.log(inc) - Math.log(100)) / (Math.log(75000) - Math.log(100));

// 4. Composite HDI (Geometric Mean of all three dimensions)
const geometricComposite = Math.cbrt(
  Math.max(0.0001, healthIndex) * Math.max(0.0001, educationIndex) * Math.max(0.0001,
incomeIndex)
);

// Identify development tier based on UN classifications
let tier = "Low Human Development";
if (geometricComposite >= 0.800) {
  tier = "Very High Human Development";
} else if (geometricComposite >= 0.700) {
  tier = "High Human Development";
} else if (geometricComposite >= 0.550) {
  tier = "Medium Human Development";
}

// Render recalculated values back to UI
idxHealth.innerText = healthIndex.toFixed(3);
idxEducation.innerText = educationIndex.toFixed(3);
idxIncome.innerText = incomeIndex.toFixed(3);
compositeScore.innerText = geometricComposite.toFixed(3);
developmentTier.innerText = tier;
}

// Handles country changes to retrieve baseline data from backend
countrySelect.addEventListener("change", async (e) => {
  const iso3 = e.target.value;
  if (!iso3) return;
```

```

try {
  const res = await fetch(`/api/country/${iso3}`);
  const data = await res.json();
  if (data.error) return;

  // Load values into sliders
  slideHealth.value = data.baselines.life_expectancy;
  slideExpSchool.value = data.baselines.expected_schooling;
  slideMeanSchool.value = data.baselines.mean_schooling;
  slideIncome.value = data.baselines.gni_per_capita;

  updateSimulation();
  renderTrendChart(data.historical_years, data.historical_scores);
} catch (err) {
  console.error("Error connecting with backend:", err);
}
});

// Attach real-time input change listeners to sliders
[slideHealth, slideExpSchool, slideMeanSchool, slideIncome].forEach(slider => {
  slider.addEventListener("input", updateSimulation);
});

function renderTrendChart(labels, values) {
  if (myChart) myChart.destroy();
  const ctx = document.getElementById("historicalChart").getContext("2d");
  myChart = new Chart(ctx, {
    type: 'line',
    data: {
      labels: labels,
      datasets: [{
        label: 'Historical Human Development Index Trend',
        data: values,
        borderColor: '#0ea5e9',
        backgroundColor: 'rgba(14, 165, 233, 0.1)',
        borderWidth: 2.5,
        fill: true,
        tension: 0.3
      }]
    },
    options: {
      responsive: true,
      maintainAspectRatio: false,
      scales: {
        y: { min: 0.3, max: 1.0 }
      }
    }
  });
}
});

```

MILESTONE 5 — VALIDATION & SYSTEM TESTING

Validation and testing are executed to ensure that mathematical outputs align exactly with official UN guidelines, UI bindings remain stable, and backend queries perform correctly.

Validation Framework & Tests

1. **Recalculation Unit Tests:** These verify that the math engine correctly handles logarithmic limits, caps maximum boundary parameters, and avoids division-by-zero errors.
2. **API Access Validation:** Checks endpoint status codes (200 OK for valid codes, 404 Not Found for invalid inputs).
3. **Cross-Browser Verification:** Ensures that visual grids, dropdown selections, and sliders behave consistently on multiple desktop and mobile browsers.

Python

Sample testing script to run metric validation checks

```
import unittest
```

```
import numpy as np
```

```
def compute_hdi(life_exp, exp_school, mean_school, gni):
```

```
    health_idx = (life_exp - 20) / (85 - 20)
```

```
    edu_idx = ((exp_school / 18) + (mean_school / 15)) / 2
```

```
    income_idx = (np.log(gni) - np.log(100)) / (np.log(75000) - np.log(100))
```

```
    return np.cbrt(health_idx * edu_idx * income_idx)
```

```
class TestHDICalculations(unittest.TestCase):
```

```
    def test_standard_high_income(self):
```

```
        # Verification check: high life expectancy, education, and wealth
```

```
        calculated = compute_hdi(81.2, 16.5, 12.5, 45000)
```

```
        self.assertTrue(0.800 <= calculated <= 1.0) # Must fall into Very High tier
```

```
    def test_minimum_boundaries(self):
```

```
        # Check that metrics do not drop below 0 if values fall near historical absolute minimums
```

```
        calculated = compute_hdi(21.0, 1.0, 1.0, 110)
```

```
        self.assertTrue(calculated > 0.0)
```

```
if __name__ == '__main__':
```

```
    unittest.main()
```

MILESTONE 6 — CONCLUSION & ROADMAP

Conclusion

The **Human Development Index (HDI) Explorer & Simulator** is a useful tool for analyzing historical progress and evaluating policy impact. By utilizing interactive slider simulations and real-time calculations, it makes complex development datasets accessible to users. Integrating backend routing with lightweight client-side calculations ensures fast performance, making the platform a practical decision-support system for policy planners.

Future Scope

- **Dynamic Dataset Ingestion:** We plan to integrate official, real-time UN Data APIs to automatically load updated annual reports and indicator sets as they are published.
- **Predictive AI Modeling:** Future updates will introduce regression and time-series forecasts to project a country's next 10 years of development based on historical policy spending.
- **Country Comparison Dashboard:** Allow users to plot multiple countries side-by-side to compare historical trajectories and simulated policy scenarios.